

INSTRUCTIONS:

This is a template folder. Make a copy of “plugin-workflow” to your plugin projects folder and rename it as per your plugin name.

STEPS:

Fill in PluginSpec.md file with all the details before you do anything. Every field!

This is the single document every skill reads from. Leaving it half-empty means every phase downstream will ask you the same questions back. Think and do it slowly and in detail. DO NOT start till you have the concept ready.

CLAUDE.md - the project identity table

Plugin name

Plugin code

Manufacturer code

JUCE path,

Project root

Etc.. All details.

Fill in everything and go through “Getting Started” document.

Why are skills hidden inside .claude?

Cmd+shift+. (period) in Finder to view hidden files.

Discovery, not preference. Claude Code automatically scans `.claude/skills/` in your project root at the start of a session — that's the mechanism by which it knows what skills exist at all. If skills lived in a plain `skills/` folder at the top level, Claude Code wouldn't auto-discover them; you'd have to manually point to each one every time, which defeats the purpose of having a skill system instead of just pasting instructions into chat.

The dot-prefix convention signals "tooling, not content." This mirrors `.github/` for GitHub Actions, `.vscode/` for editor settings, `.git/` for version control internals. It's a long-standing convention across dev tooling: dotfiles/dotfolders are configuration and infrastructure for a tool, not part of the project's actual deliverable. Your plugin's C++ source, your DSP research findings, your reports — those are the *content*. The skills are *instructions that shape how Claude works on that content*. Putting them in `.claude/` keeps that boundary visually obvious in a file browser and keeps them out of the way when you're just looking at your actual project files.

It also keeps them out of casual version-control noise and IDE search. Most file explorers and many IDEs collapse or hide dotfolders by default, and `.gitignore`-style tooling treats them as a separate category. You generally do want to commit `.claude/skills/` to git (so the whole team/you-on-another-machine gets the same skills), but you don't want it cluttering every file search you do while writing actual code.

So it's not arbitrary — it's the specific path Claude Code's discovery mechanism looks for, following an established convention for "this is tool configuration, not project content." If you renamed it to plain `skills/`, Claude Code simply wouldn't find it without being told explicitly every session.